

Compiler Design and Construction Foundations

Chapter 1: The Structure of a Modern Compiler

A compiler translates source code written in a high-level language into machine code or bytecode. The compilation pipeline is divided into Analysis (Front End) and Synthesis (Back End).

1. Lexical Analysis (Scanning):

Converts a sequence of characters into a sequence of meaningful tokens (keywords, identifiers, literals, operators). Regular expressions define tokens, and Finite State Automata (DFA/NFA) recognize them.

2. Syntax Analysis (Parsing):

Imposes a hierarchical structure on tokens according to Context-Free Grammars (CFG).

Outputs a Parse Tree or Abstract Syntax Tree (AST).

Top-Down parsers (LL) and Bottom-Up parsers (LR, LALR, SLR) resolve language ambiguities.

3. Semantic Analysis:

Performs type checking, scope resolution, and ensures logical consistency across declarations and uses.

4. Intermediate Code Generation (IR):

Generates three-address code (TAC) or Static Single Assignment (SSA) form for machine-independent optimization.

5. Code Optimization:

Eliminates dead code, performs loop invariant code motion, constant folding, and register allocation.

6. Code Generation:

Translates optimized IR into target assembly instructions for x86-64 or ARM architectures.